K.R. MANGALAM UNIVERSITY

SMART CITY TRAFFIC MANAGEMENT SYSTEM

DAA Capstone Project — Final Report

Student Name	Harshit Kumar
Roll Number	2401201057
Programme	BCA (AI & Data Science)
Semester	V
Course Code	ENCA351
Course Name	Design and Analysis of Algorithms Lab
Faculty	Dr. Aarti
Academic Session	2026–27

Route Planning • Traffic Analysis • Connectivity • Infrastructure Optimization

1. Abstract

The Smart City Traffic Management System is a Python-based educational prototype demonstrating how graph algorithms support road-network planning. Intersections are vertices and roads are weighted edges. The Streamlit application provides road-network management, shortest-route planning, simulated traffic changes, connectivity analysis, minimum spanning tree planning and algorithm benchmarking.

The screenshots in this report show the current application state, including 7 intersections and 13 roads on the dashboard. The benchmarking module reports execution time and peak memory for selected graph algorithms.

2. Introduction and Problem Statement

Urban road networks contain intersections connected by roads with different travel costs. Congestion and road closures can change which route is most efficient. Graph theory models intersections as vertices, roads as edges and road weights as travel or infrastructure costs.

Problem statement: Develop an interactive prototype to model a city road network and demonstrate shortest-route planning, simulated congestion analysis, network connectivity, minimum spanning tree planning and algorithm-performance comparison.

3. Objectives

• Represent intersections and roads using a weighted graph.

• Calculate a minimum-cost route between a selected source and destination.

• Simulate congestion and road disruption by changing road costs or availability.

• Use BFS and DFS to inspect traversal order and reachability.

• Calculate an MST using Prim’s and Kruskal’s algorithms.

• Compare execution time and peak memory across graph algorithms.

• Present results through an interactive Streamlit interface.

4. Technologies Used

Technology	Purpose
Python	Core programming language and algorithm integration.
Streamlit	Interactive browser interface, navigation, forms and results.
NetworkX	Graph representation and operations.
Pandas	Benchmark tables and CSV data handling.
Matplotlib	Graph and performance visualizations.
Jupyter Notebook	Algorithm demonstrations and analysis.
Git / GitHub	Version control and project hosting.

5. System Design and Modules

The application is organized into separate files so the user interface and algorithm logic remain manageable. The Streamlit app provides navigation and input controls, while helper modules perform graph operations.

File	Responsibility
app.py	Streamlit navigation, input controls, graph rendering and results.
route_planner.py	Shortest-route calculation using Dijkstra's algorithm.
traffic_analysis.py	Route and cost comparison after simulated traffic changes.
connectivity_module.py	Connectivity checks, BFS/DFS traversal and connected components.
mst_planner.py	Minimum Spanning Tree using Prim or Kruskal.
benchmark.py	Algorithm execution-time and memory benchmarking.
project_notebook.ipynb	Algorithm demonstrations and experimental analysis.
requirements.txt	Python package dependencies.
README.md	Project overview and setup instructions.

6. Algorithms Implemented

Algorithm	Purpose	Typical complexity
Dijkstra	Minimum-cost route with non-negative weights.	O((V + E) log V)
Bellman–Ford	Shortest paths; supports negative weights if no negative cycle exists.	O(VE)
BFS	Breadth-first traversal and reachability.	O(V + E)
DFS	Depth-first traversal and connectivity exploration.	O(V + E)
Prim	Minimum Spanning Tree using low-cost connecting edges.	O(E log V)
Kruskal	MST by considering edges in ascending weight order.	O(E log E)

V = number of vertices; E = number of edges. Actual performance depends on implementation and graph structure.

7. Application Workflow

• Load the default city graph or add intersections and roads.

• Choose a module from the application navigation.

• Select the required source, destination, road or algorithm.

• Execute the operation and inspect the displayed result.

• Review graph visualizations and export benchmark results when required.

8. Application Screenshots and Outputs

The following figures are captured from the running Streamlit application and demonstrate its main modules.

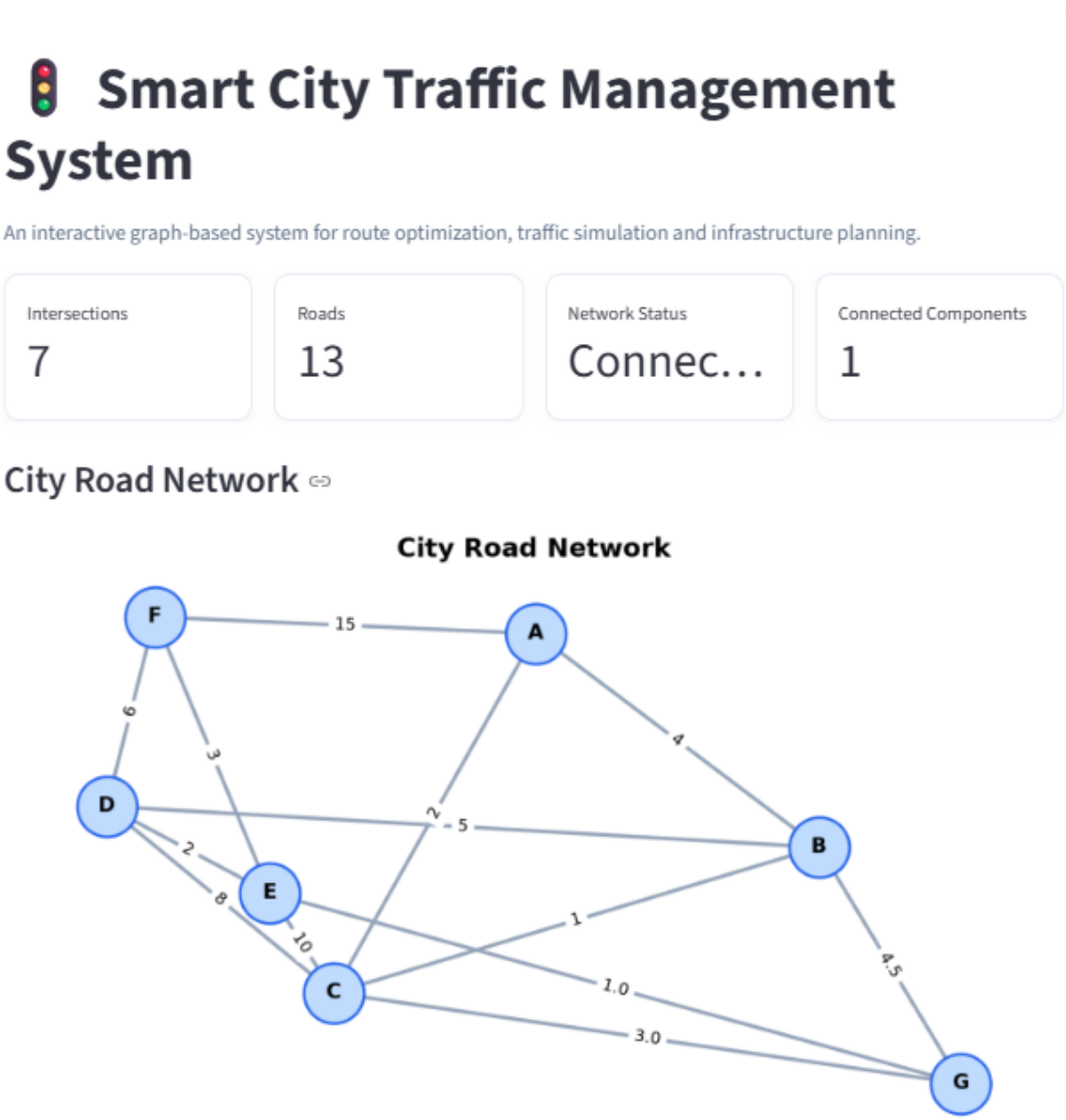


Figure 1. Dashboard: 7 intersections, 13 roads and one connected component.

Deploy

Road Network Management

Add intersections and roads to customize the city graph.

Intersection name

Example: G

+ Add Intersection

Connect two intersections

Starting intersection

A

Ending intersection

B

Road cost / distance

1.00

+ Add / Update Road

Current Road Network

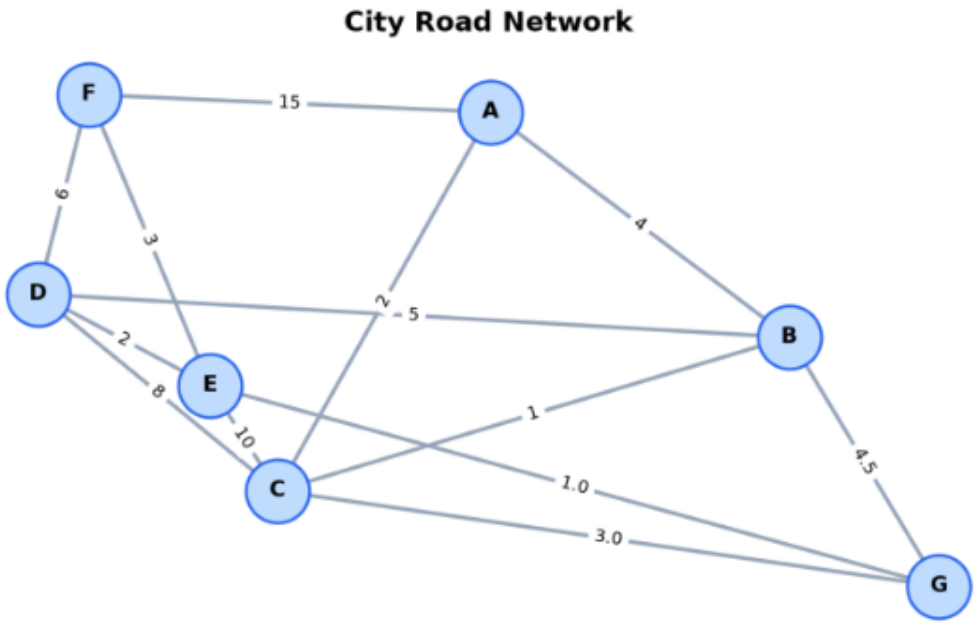


Figure 2. Road Network Management: add intersections and connect roads.

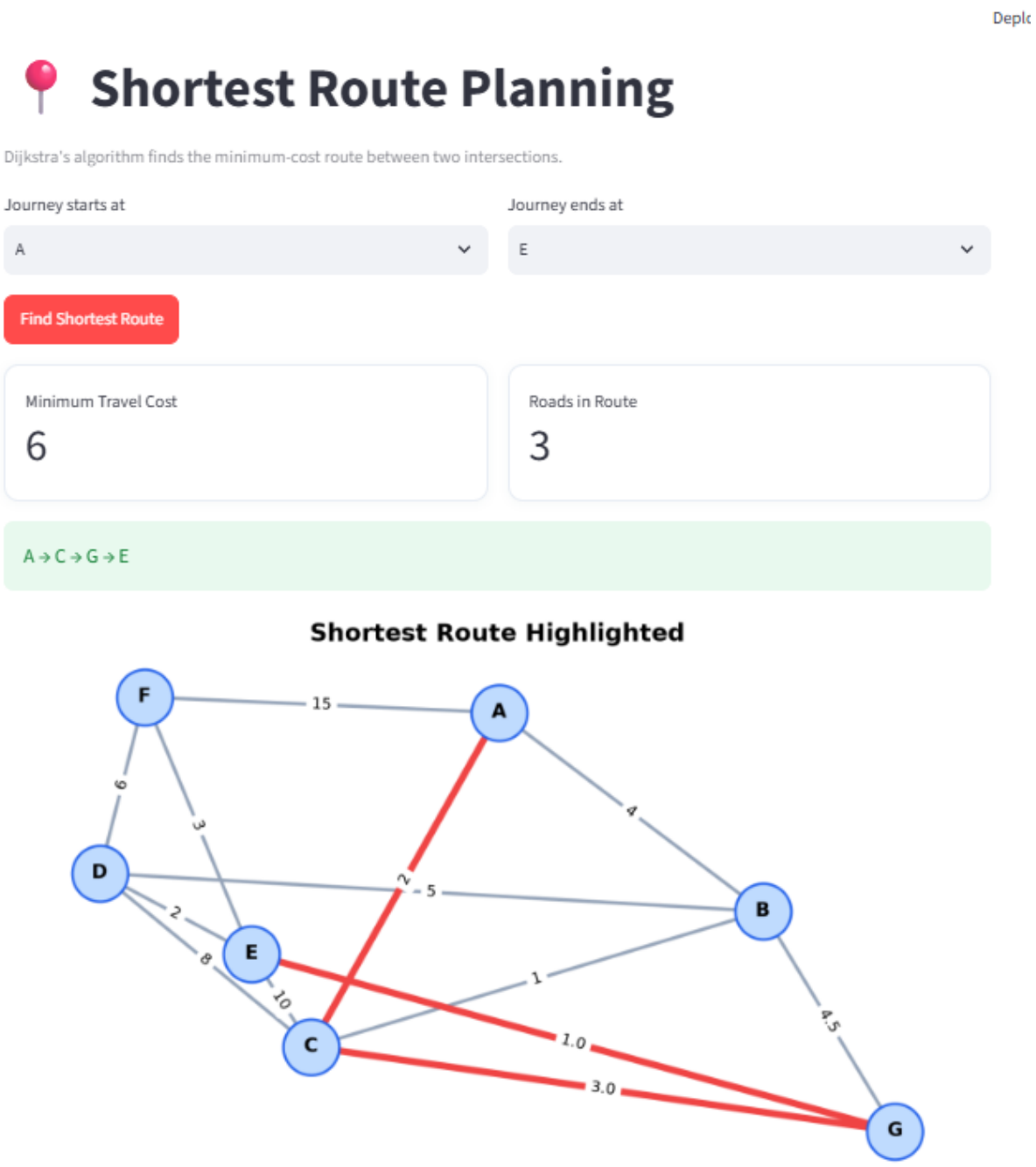


Figure 3. Dijkstra route example: A → C → G → E; minimum travel cost 6.

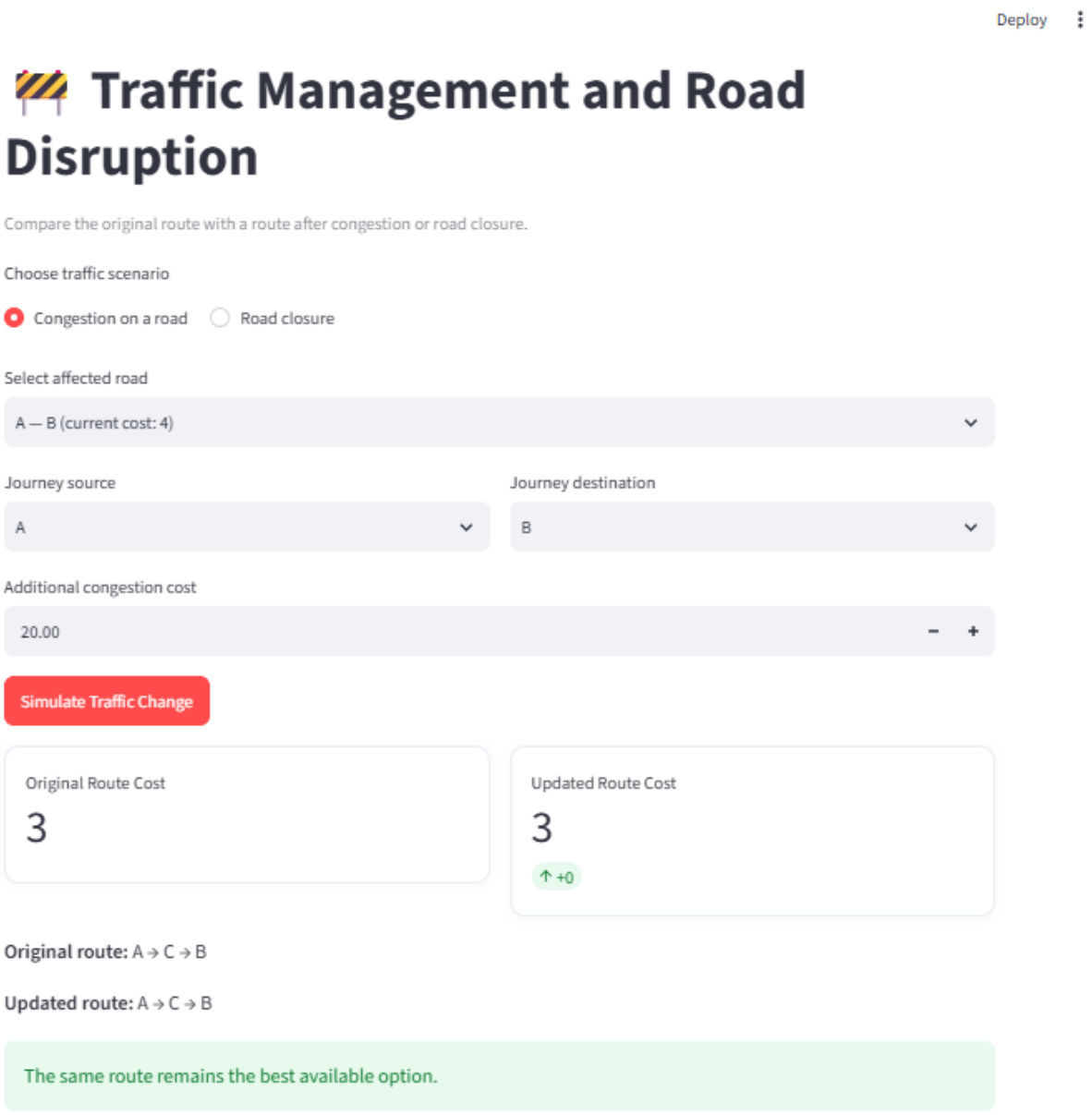


Figure 4. Traffic congestion simulation with original and updated route costs.

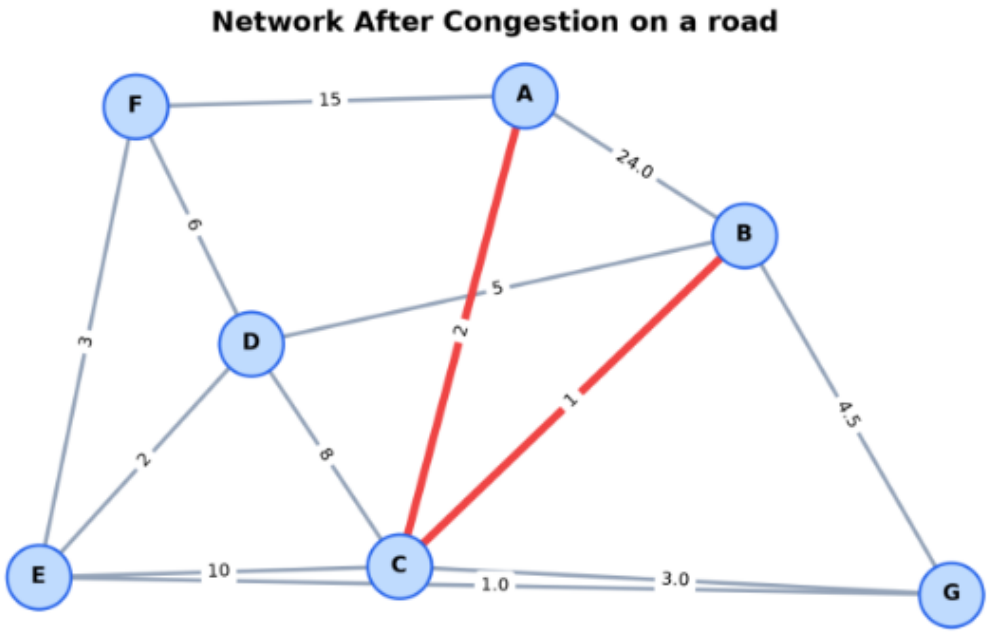


Figure 5. Road-network visualization after a traffic-cost change.

Deploy

🔗 Network Connectivity

Explore graph traversal and connectivity using BFS and DFS.

Network Connected	Connected Components	Total Intersections
Yes	1	7

Traversal starting intersection

A

Run BFS | Run DFS

BFS Traversal

A → B → C → F → D → G → E

Intersections reached: 7

Connected Components

Component 1: A, B, C, D, E, F, G

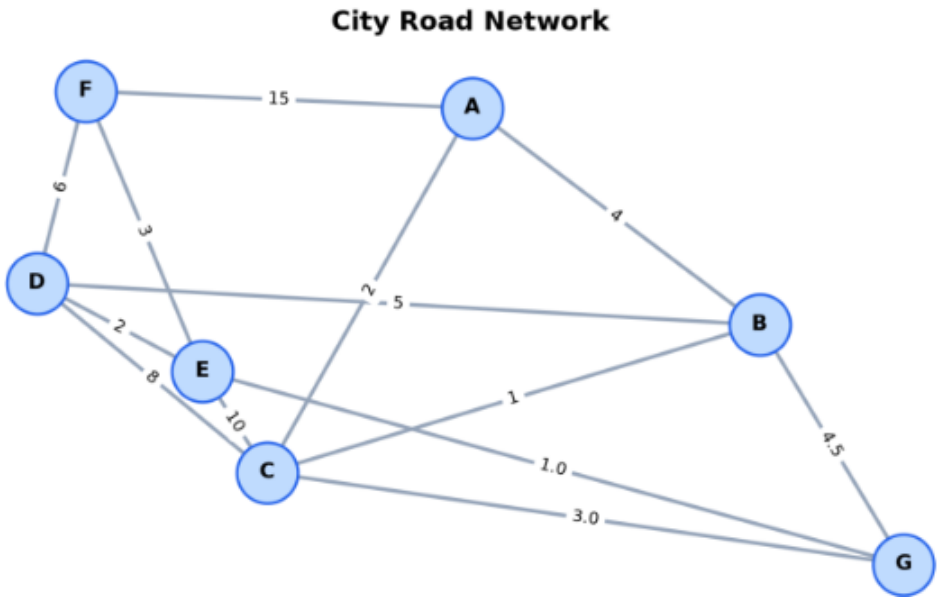


Figure 6. BFS traversal and connected-components output.

De

Network Connectivity

Explore graph traversal and connectivity using BFS and DFS.

Network Connected	Connected Components	Total Intersections
Yes	1	7

Traversal starting intersection

F

Run BFS | Run DFS

DFS Traversal

F → D → B → A → C → E → G

Intersections reached: 7

Connected Components

Component 1: A, B, C, D, E, F, G

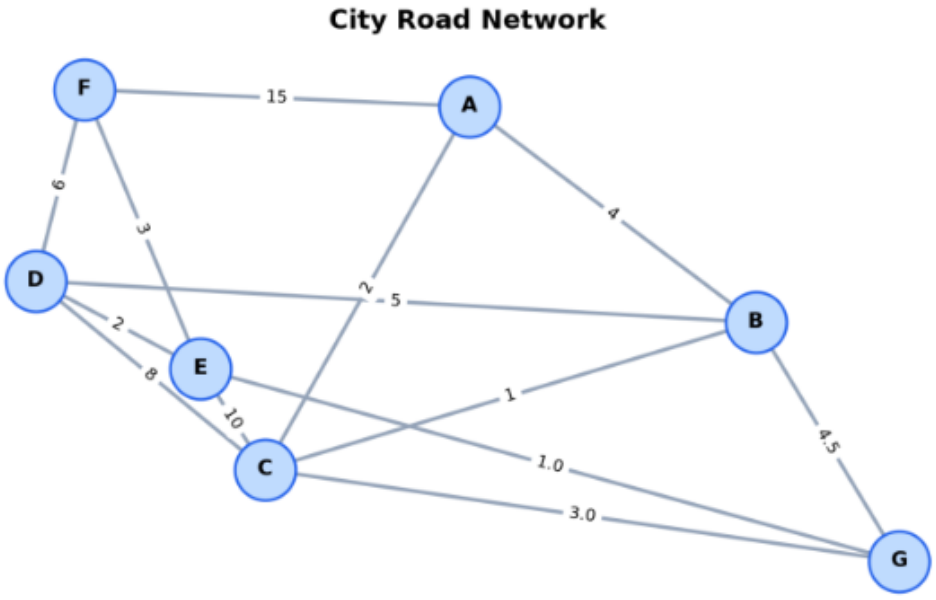


Figure 7. DFS traversal from the selected starting intersection.

Deploy

Infrastructure Planning

Find a Minimum Spanning Tree using Prim's or Kruskal's algorithm.

Select algorithm

Prim

Total Infrastructure Cost

12

Selected Roads

Road	Cost
A — C	2
B — C	1
C — G	3
D — E	2
E — G	1
E — F	3

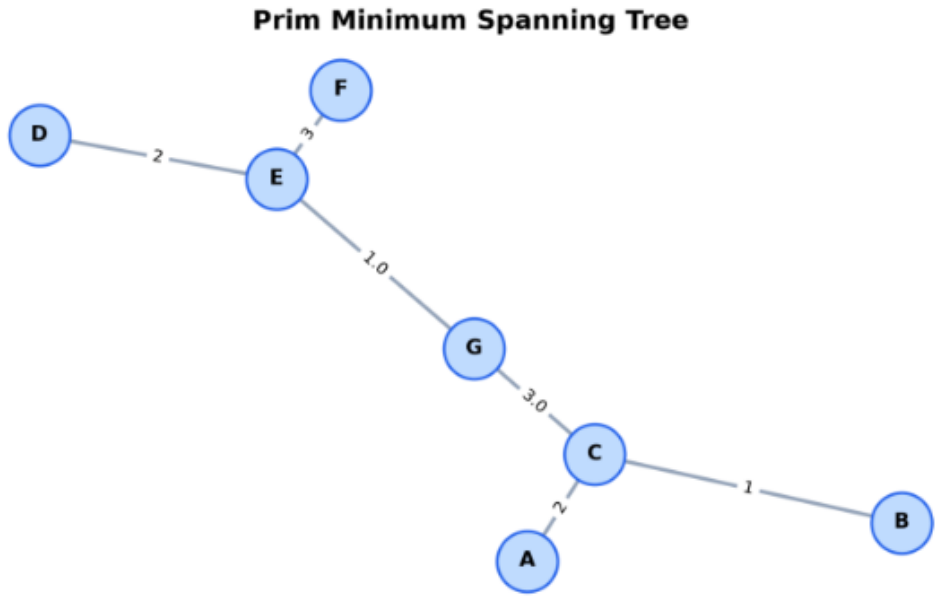


Figure 8. Prim's Minimum Spanning Tree; displayed total infrastructure cost 12.

Deploy

Infrastructure Planning

Find a Minimum Spanning Tree using Prim's or Kruskal's algorithm.

Select algorithm

Kruskal

Total Infrastructure Cost

12

Selected Roads

Road	Cost
A — C	2
B — C	1
C — G	3
D — E	2
E — G	1
E — F	3

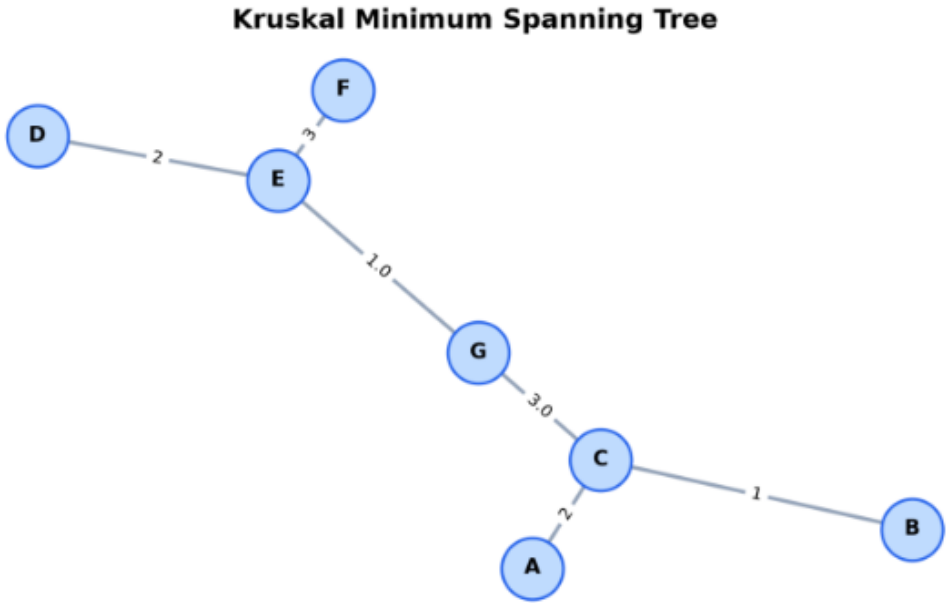


Figure 9. Kruskal's Minimum Spanning Tree; displayed total infrastructure cost 12.

Algorithm Performance Analysis

Compare execution time for BFS, DFS, Dijkstra, Bellman-Ford, Prim and Kruskal.

The benchmark tests networks with different numbers of intersections. Execution time can vary between runs and depends on the computer.

Run Benchmark

Benchmark completed.

Intersections	Roads	Algorithm	Execution Time (ms)	Peak Memory (KB)
10	30	BFS	23.1279	57.95
10	30	DFS	2.9511	48.75
10	30	Dijkstra	3.8636	53.21
10	30	Bellman-Ford	4.9345	51.04
10	30	Prim	0.764	9.76
10	30	Kruskal	2.4104	45.48
25	75	BFS	0.605	20.01
25	75	DFS	0.5551	21.62
25	75	Dijkstra	0.2898	4.49
25	75	Bellman-Ford	0.7572	8.71

Figure 10. Benchmark table with execution-time and peak-memory measurements.

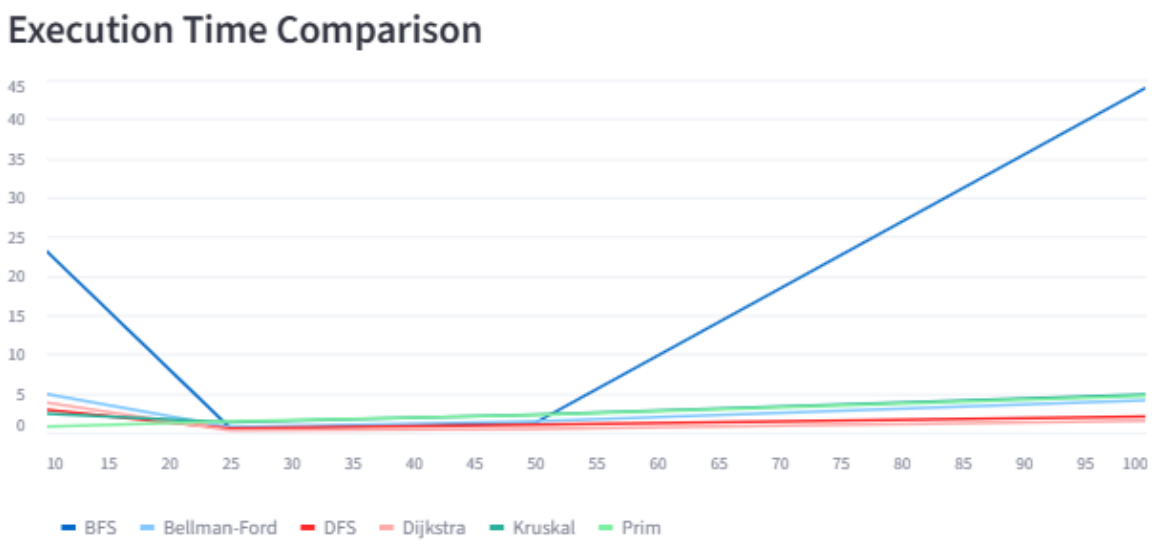


Figure 11. Execution-time comparison chart.

9. Results and Testing

The uploaded application screenshots provide evidence of the following visible results:

Module	Observed result	Demonstration
Dashboard	7 intersections, 13 roads, 1 connected component.	Current network overview.
Shortest route	A → C → G → E; cost 6; 3 roads.	Dijkstra route planning and highlighting.
Traffic management	Original and updated route costs are displayed.	Traffic-cost change comparison.
Connectivity	Connected = Yes; 1 component; 7 intersections.	BFS/DFS and connectivity status.
Prim / Kruskal	Both display total infrastructure cost 12.	MST planning using two algorithms.
Benchmarking	Execution time and peak-memory columns are shown.	Algorithm performance measurement.

10. Performance Analysis

The benchmarking module compares BFS, DFS, Dijkstra, Bellman–Ford, Prim and Kruskal on generated graphs of different sizes. The visible benchmark table includes intersection count, road count, algorithm, execution time in milliseconds and peak memory in kilobytes. A chart visualizes execution-time trends.

Benchmark values can vary between runs because of system load, interpreter overhead and the short duration of operations on small graphs. They should be treated as measurements from the current environment rather than universal rankings. Repeated trials on identical graph instances would provide a more robust comparison.

11. Limitations and Future Enhancements

Current limitations

• The network is illustrative and does not use real geographic coordinates.

• Road weights represent generalized costs rather than calibrated live travel times.

• Traffic changes are simulated manually rather than obtained from a live traffic service.

• Benchmark results depend on the local computer and generated graph instances.

Future enhancements

• Integrate real road-network data and map-based visualization.

• Connect to live traffic services to update road costs dynamically.

• Store network data and benchmark runs in a database.

• Repeat benchmark trials and report average, median and standard deviation.

• Add automated tests for disconnected graphs and invalid inputs.

12. Conclusion

The Smart City Traffic Management System demonstrates how graph algorithms can support route planning, simulated traffic analysis, connectivity inspection and infrastructure planning. The Streamlit interface brings these tasks together in an interactive prototype, while benchmarking provides execution-time and memory comparisons. The submitted screenshots show the application’s principal features and outputs. The project provides a foundation for future work using real road data and dynamic traffic costs.

13. References

• Python Documentation — https://docs.python.org/

• Streamlit Documentation — https://docs.streamlit.io/

• NetworkX Documentation — https://networkx.org/documentation/stable/

• Pandas Documentation — https://pandas.pydata.org/docs/

• Matplotlib Documentation — https://matplotlib.org/stable/

Appendix A — Application Source Code

File: app.py | Main Streamlit application

```
001 import streamlit as st
002 import networkx as nx
003 import matplotlib.pyplot as plt
004 import pandas as pd
005
006 from route_planner import find_shortest_route
007 from traffic_analysis import analyze_traffic
008 from benchmark import benchmark_algorithms
009 from connectivity_module import (
010     check_connectivity,
011     get_bfs_traversal,
012     get_dfs_traversal,
013     get_connected_components,
014 )
015 from mst_planner import find_mst
016
017
018 # -------------------- PAGE CONFIG --------------------
019 st.set_page_config(
020     page_title="Smart City Traffic Management",
021     page_icon="■",
022     layout="wide",
023     initial_sidebar_state="expanded",
024 )
025
026 # -------------------- CUSTOM STYLE --------------------
027 st.markdown(
028     """
029     <style>
030     .main {background-color: #f7f9fc;}
031     [data-testid="stMetric"] {
032         background: white;
033         border: 1px solid #e8edf5;
034         padding: 16px;
035         border-radius: 12px;
036         box-shadow: 0 2px 8px rgba(20, 40, 80, 0.04);
037     }
038     .block-container {padding-top: 1.6rem; padding-bottom: 2rem;}
039     .stButton > button {
040         border-radius: 8px;
041         font-weight: 600;
042         min-height: 2.5rem;
043     }
044     .section-note {
045         color: #64748b;
046         font-size: 0.92rem;
047     }
048     </style>
049     """,
050     unsafe_allow_html=True,
051 )
052
053
054 # -------------------- CITY NETWORK --------------------
055 def create_city():
056     graph = nx.Graph()
057     roads = [
058         ("A", "B", 4),
059         ("A", "C", 2),
060         ("B", "C", 1),
061         ("B", "D", 5),
062         ("C", "D", 8),
063         ("C", "E", 10),
064         ("D", "E", 2),
065         ("D", "F", 6),
066         ("E", "F", 3),
067         ("A", "F", 15),
068     ]
069     graph.add_weighted_edges_from(roads, weight="weight")
070     return graph
071
072
073 if "city" not in st.session_state:
074     st.session_state.city = create_city()
075
076 G = st.session_state.city
077
078
```

Appendix A — Application Source Code

File: app.py | Main Streamlit application

```
079 # -------------------- GRAPH VISUALIZATION --------------------
080 def show_graph(graph, highlighted_edges=None, title=None):
081     if graph.number_of_nodes() == 0:
082         st.info("The road network is empty. Add intersections to get started.")
083         return
084 
085     highlighted_edges = highlighted_edges or []
086     pos = nx.spring_layout(graph, seed=42)
087 
088     fig, ax = plt.subplots(figsize=(10, 5.5))
089     nx.draw_networkx_nodes(
090         graph, pos, node_color="#bfdbfe", edgecolors="#2563eb",
091         node_size=1050, linewidths=1.5, ax=ax
092     )
093     nx.draw_networkx_labels(
094         graph, pos, font_size=11, font_weight="bold", ax=ax
095     )
096 
097     normal_edges = [
098         edge for edge in graph.edges()
099         if edge not in highlighted_edges
100         and (edge[1], edge[0]) not in highlighted_edges
101     ]
102     nx.draw_networkx_edges(
103         graph, pos, edgelist=normal_edges, edge_color="#94a3b8",
104         width=2, ax=ax
105     )
106     if highlighted_edges:
107         nx.draw_networkx_edges(
108             graph, pos, edgelist=highlighted_edges,
109             edge_color="#ef4444", width=4, ax=ax
110         )
111 
112     labels = nx.get_edge_attributes(graph, "weight")
113     nx.draw_networkx_edge_labels(
114         graph, pos, edge_labels=labels, font_size=9, ax=ax
115     )
116     ax.set_title(title or "City Road Network", fontsize=14, fontweight="bold")
117     ax.axis("off")
118     st.pyplot(fig, use_container_width=True)
119     plt.close(fig)
120 
121 
122 def safe_connected(graph):
123     return graph.number_of_nodes() > 0 and nx.is_connected(graph)
124 
125 
126 def format_path(path):
127     return " → ".join(map(str, path))
128 
129 
130 # -------------------- SIDEBAR --------------------
131 st.sidebar.title("■ Smart City")
132 st.sidebar.caption("Traffic and Road Network Control")
133 page = st.sidebar.radio(
134     "NAVIGATION",
135     [
136         "Dashboard",
137         "Road Network",
138         "Route Planning",
139         "Traffic Management",
140         "Network Connectivity",
141         "Infrastructure Planning",
142         "Performance Analysis",
143     ],
144 )
145 st.sidebar.divider()
146 st.sidebar.metric("Intersections", G.number_of_nodes())
147 st.sidebar.metric("Roads", G.number_of_edges())
148 if st.sidebar.button("Reset City Network", use_container_width=True):
149     st.session_state.city = create_city()
150     st.rerun()
151 
152 
153 # -------------------- DASHBOARD --------------------
154 if page == "Dashboard":
155     st.title("■ Smart City Traffic Management System")
156     st.markdown(
```

Appendix A — Application Source Code

File: app.py | Main Streamlit application

```
        '<p class="section-note">An interactive graph-based system for route '
        'optimization, traffic simulation and infrastructure planning.</p>',
        unsafe_allow_html=True,
    )

    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Intersections", G.number_of_nodes())
    c2.metric("Roads", G.number_of_edges())
    c3.metric("Network Status", "Connected" if safe_connected(G) else "Disconnected")
    c4.metric("Connected Components", len(get_connected_components(G)) if G else 0)

    st.subheader("City Road Network")
    show_graph(G)

    left, right = st.columns(2)
    with left:
        st.info(
            "**Route Planning**\n\nFind the minimum-cost path between two "
            "intersections using Dijkstra's algorithm."
        )
        st.info(
            "**Traffic Management**\n\nSimulate congestion or a road closure "
            "and compare the route before and after the change."
        )
    with right:
        st.info(
            "**Network Connectivity**\n\nExplore BFS, DFS and connected "
            "components in the city network."
        )
        st.info(
            "**Infrastructure Planning**\n\nUse Prim's or Kruskal's algorithm "
            "to find a minimum spanning tree."
        )


# -------------------- ROAD NETWORK --------------------
elif page == "Road Network":
    st.title("■■ Road Network Management")
    st.caption("Add intersections and roads to customize the city graph.")

    with st.form("node_form", clear_on_submit=True):
        new_node = st.text_input("Intersection name", placeholder="Example: G")
        add_node = st.form_submit_button("■ Add Intersection")

    if add_node:
        new_node = new_node.strip()
        if not new_node:
            st.warning("Please enter an intersection name.")
        elif new_node in G:
            st.warning("This intersection already exists.")
        else:
            G.add_node(new_node)
            st.success(f"Intersection {new_node} added.")
            st.rerun()

    nodes = sorted(G.nodes(), key=str)
    st.subheader("Connect two intersections")
    if len(nodes) >= 2:
        with st.form("road_form", clear_on_submit=True):
            col1, col2, col3 = st.columns(3)
            with col1:
                source = st.selectbox("Starting intersection", nodes)
            with col2:
                destination = st.selectbox("Ending intersection", nodes, index=1)
            with col3:
                weight = st.number_input("Road cost / distance", min_value=0.1, value=1.0, step=0.5)
            add_road = st.form_submit_button("■ Add / Update Road")

        if add_road:
            if source == destination:
                st.warning("Select two different intersections.")
            else:
                G.add_edge(source, destination, weight=float(weight))
                st.success(f"Road {source} — {destination} saved.")
                st.rerun()
    else:
        st.info("Add at least two intersections before connecting roads.")

```

Appendix A — Application Source Code

File: app.py | Main Streamlit application

```
    st.subheader("Current Road Network")
    show_graph(G)

    if G.number_of_edges():
        road_rows = [
            {"Starting Point": u, "Ending Point": v, "Cost": data.get("weight", 1)}
            for u, v, data in G.edges(data=True)
        ]
        st.dataframe(pd.DataFrame(road_rows), use_container_width=True, hide_index=True)

        with st.expander("Remove a road"):
            road_options = list(G.edges())
            selected_road = st.selectbox(
                "Choose road",
                road_options,
                format_func=lambda edge: f"{edge[0]} — {edge[1]}",
            )
            if st.button("Remove Selected Road"):
                G.remove_edge(*selected_road)
                st.success("Road removed.")
                st.rerun()

    with st.expander("Network Data: Adjacency List and Matrix"):
        st.write("**Adjacency List**")
        st.json({
            str(node): {
                str(neighbor): data.get("weight", 1)
                for neighbor, data in G[node].items()
            }
            for node in G.nodes()
        })
        st.write("**Adjacency Matrix**")
        st.dataframe(nx.to_pandas_adjacency(G, weight="weight"), use_container_width=True)


# -------------------- ROUTE PLANNING --------------------
elif page == "Route Planning":
    st.title("■ Shortest Route Planning")
    st.caption("Dijkstra's algorithm finds the minimum-cost route between two intersections.")

    nodes = sorted(G.nodes(), key=str)
    if len(nodes) < 2:
        st.warning("Add at least two intersections in Road Network.")
    else:
        col1, col2 = st.columns(2)
        with col1:
            source = st.selectbox("Journey starts at", nodes, key="route_source")
        with col2:
            destination = st.selectbox("Journey ends at", nodes, key="route_destination")

        if st.button("Find Shortest Route", type="primary"):
            if source == destination:
                st.warning("Starting point and destination must be different.")
            else:
                route, distance = find_shortest_route(G, source, destination)
                if route is not None:
                    m1, m2 = st.columns(2)
                    m1.metric("Minimum Travel Cost", f"{distance:g}")
                    m2.metric("Roads in Route", max(0, len(route) - 1))
                    st.success(format_path(route))
                    show_graph(
                        G,
                        highlighted_edges=list(zip(route, route[1:])),
                        title="Shortest Route Highlighted",
                    )
                    st.write("**Route steps**")
                    for index, location in enumerate(route, start=1):
                        st.write(f"{index}. {location}")
                else:
                    st.error("No route exists between these intersections.")


# -------------------- TRAFFIC MANAGEMENT --------------------
elif page == "Traffic Management":
    st.title("■ Traffic Management and Road Disruption")
    st.caption("Compare the original route with a route after congestion or road closure.")

    nodes = sorted(G.nodes(), key=str)
```

Appendix A — Application Source Code

File: app.py | Main Streamlit application

```
    edges = list(G.edges(data=True))

    if len(nodes) < 2 or not edges:
        st.warning("The network needs at least two intersections and one road.")
    else:
        mode = st.radio(
            "Choose traffic scenario",
            ["Congestion on a road", "Road closure"],
            horizontal=True,
        )
        road_index = st.selectbox(
            "Select affected road",
            range(len(edges)),
            format_func=lambda i: (
                f"{edges[i][0]} — {edges[i][1]} "
                f"(current cost: {edges[i][2].get('weight', 1)})"
            ),
        )
        source_col, destination_col = st.columns(2)
        with source_col:
            source = st.selectbox("Journey source", nodes, key="traffic_source")
        with destination_col:
            destination = st.selectbox("Journey destination", nodes, key="traffic_destination")

        extra_cost = 0.0
        if mode == "Congestion on a road":
            extra_cost = st.number_input(
                "Additional congestion cost",
                min_value=0.0,
                value=5.0,
                step=1.0,
            )

        if st.button("Simulate Traffic Change", type="primary"):
            if source == destination:
                st.warning("Choose different starting and ending intersections.")
            else:
                u, v, data = edges[road_index]
                original_path, original_cost = find_shortest_route(G, source, destination)

                changed_graph = G.copy()
                if mode == "Congestion on a road":
                    changed_graph[u][v]["weight"] += extra_cost
                else:
                    changed_graph.remove_edge(u, v)

                if original_path is None:
                    st.error("No original route exists for this journey.")
                else:
                    if nx.has_path(changed_graph, source, destination):
                        changed_path = nx.dijkstra_path(
                            changed_graph, source, destination, weight="weight"
                        )
                        changed_cost = nx.dijkstra_path_length(
                            changed_graph, source, destination, weight="weight"
                        )
                        a, b = st.columns(2)
                        a.metric("Original Route Cost", f"{original_cost:g}")
                        b.metric("Updated Route Cost", f"{changed_cost:g}",
delta=f"{changed_cost-original_cost:+g}")
                        st.write("**Original route:**", format_path(original_path))
                        st.write("**Updated route:**", format_path(changed_path))
                        st.success(
                            "An alternate route is available."
                            if changed_path != original_path
                            else "The same route remains the best available option."
                        )
                        show_graph(
                            changed_graph,
                            highlighted_edges=list(zip(changed_path, changed_path[1:])),
                            title=f"Network After {mode}",
                        )
                    else:
                        st.error("No alternate route is available after this change.")
                        st.write("Original route:", format_path(original_path))
                        show_graph(changed_graph, title="Network After Road Disruption")


```

Appendix A — Application Source Code

File: app.py | Main Streamlit application

```
390 # -------------------- CONNECTIVITY --------------------
391 elif page == "Network Connectivity":
392     st.title("■ Network Connectivity")
393     st.caption("Explore graph traversal and connectivity using BFS and DFS.")
394 
395     nodes = sorted(G.nodes(), key=str)
396     connected = safe_connected(G)
397     c1, c2, c3 = st.columns(3)
398     c1.metric("Network Connected", "Yes" if connected else "No")
399     c2.metric("Connected Components", len(get_connected_components(G)) if G else 0)
400     c3.metric("Total Intersections", G.number_of_nodes())
401 
402     if nodes:
403         start = st.selectbox("Traversal starting intersection", nodes)
404         bfs_col, dfs_col = st.columns(2)
405         with bfs_col:
406             if st.button("Run BFS", use_container_width=True):
407                 order = get_bfs_traversal(G, start)
408                 st.subheader("BFS Traversal")
409                 st.success(format_path(order))
410                 st.write(f"Intersections reached: {len(order)}")
411         with dfs_col:
412             if st.button("Run DFS", use_container_width=True):
413                 order = get_dfs_traversal(G, start)
414                 st.subheader("DFS Traversal")
415                 st.success(format_path(order))
416                 st.write(f"Intersections reached: {len(order)}")
417 
418     st.subheader("Connected Components")
419     components = get_connected_components(G)
420     for index, component in enumerate(components, start=1):
421         st.write(f"**Component {index}:** {', '.join(map(str, sorted(component, key=str)))}")
422     show_graph(G)
423 
424 
425 # -------------------- INFRASTRUCTURE PLANNING --------------------
426 elif page == "Infrastructure Planning":
427     st.title("■■ Infrastructure Planning")
428     st.caption("Find a Minimum Spanning Tree using Prim's or Kruskal's algorithm.")
429 
430     if G.number_of_nodes() == 0:
431         st.info("Add intersections to begin infrastructure planning.")
432     else:
433         algorithm = st.selectbox("Select algorithm", ["Prim", "Kruskal"])
434         mst, total_cost = find_mst(G, algorithm.lower())
435 
436         if not safe_connected(G):
437             st.warning(
438                 "The network is disconnected. The result is a minimum spanning "
439                 "forest for the available connected components."
440             )
441 
442         st.metric("Total Infrastructure Cost", f"{total_cost:g}")
443         st.subheader("Selected Roads")
444         mst_rows = [
445             {"Road": f"{u} — {v}", "Cost": data.get("weight", 1)}
446             for u, v, data in mst.edges(data=True)
447         ]
448         if mst_rows:
449             st.dataframe(pd.DataFrame(mst_rows), use_container_width=True, hide_index=True)
450         else:
451             st.info("No roads are available for the spanning tree.")
452         show_graph(mst, title=f"{algorithm} Minimum Spanning Tree")
453 
454 
455 # -------------------- PERFORMANCE ANALYSIS --------------------
456 elif page == "Performance Analysis":
457     st.title("■ Algorithm Performance Analysis")
458     st.caption("Compare execution time for BFS, DFS, Dijkstra, Bellman-Ford, Prim and Kruskal.")
459 
460     st.write(
461         "The benchmark tests networks with different numbers of intersections. "
462         "Execution time can vary between runs and depends on the computer."
463     )
464 
465     if st.button("Run Benchmark", type="primary"):
466         with st.spinner("Running algorithm benchmarks..."):
467             results = benchmark_algorithms()
```

Appendix A — Application Source Code

File: app.py | Main Streamlit application

```
        st.session_state.benchmark_results = results

    if "benchmark_results" in st.session_state:
        results = st.session_state.benchmark_results
        st.success("Benchmark completed.")
        st.dataframe(results, use_container_width=True, hide_index=True)

        st.subheader("Execution Time Comparison")
        chart_data = results.pivot(
            index="Intersections",
            columns="Algorithm",
            values="Execution Time (ms)",
        )
        st.line_chart(chart_data, use_container_width=True)

        st.subheader("Compare algorithms for one network size")
        sizes = sorted(results["Intersections"].unique())
        selected_size = st.selectbox("Number of intersections", sizes)
        size_data = results[results["Intersections"] == selected_size].set_index("Algorithm")
        st.bar_chart(size_data["Execution Time (ms)"], use_container_width=True)

        st.download_button(
            "■■ Download Benchmark CSV",
            data=results.to_csv(index=False).encode("utf-8"),
            file_name="smart_city_benchmark_results.csv",
            mime="text/csv",
        )
    else:
        st.info("Click **Run Benchmark** to generate the performance results.")
```

PROJECT GITHUB REPOSITORY

https://github.com/Harshitkumar0018/smart-city-traffic-system-Harshit-Kumar/tree/main

Complete project repository and supporting files